

# RAG Chatbot

## Tư Vấn Thủ Tục Nhập Học 2025

Tổng quan vận hành & Tài liệu thuyết trình

Trường Đại học Khoa học Tự nhiên - ĐHQG Hà Nội  
Năm 2025

## Mục lục

---

- 1. Tổng quan quy trình vận hành
- 2. Vai trò từng file
- 3. Luồng xử lý runtime
- 4. Quy trình chạy project lần đầu
- 5. Câu hỏi trọng tâm thuyết trình (10 câu)
- 6. Sơ đồ kiến trúc tổng thể

# 1. Tổng quan quy trình vận hành

---

Hệ thống RAG Chatbot tư vấn thủ tục nhập học hoạt động theo pipeline 5 phase:

Phase 1: Chunking tài liệu nguồn thành các đoạn nhỏ có metadata.

Phase 2: Tạo embedding vector và lưu vào Vector DB.

Phase 4: Xây dựng prompt engineering cho LLM.

Phase 5: Generate câu trả lời (deterministic + LLM fallback + naturalizer).

Khi user hỏi, hệ thống phân tích intent -> truy xuất chunks liên quan -> format câu trả lời chính xác hoặc gọi LLM -> validate -> naturalize -> trả về.

1. Chuẩn bị dữ liệu (data/)
2. Chunking (phase1)
3. Embedding + Vector DB (phase2)
4. Khởi động web server (app.py)
5. User hỏi -> Retriever -> Formatter/LLM -> Trả lời

## 2. Vai trò từng file

---

config.py

Xác định file dữ liệu nguồn và năm tuyển sinh từ .env hoặc filesystem

phase1\_chunking.py

Đọc TXT/DOCX/PDF, chia thành chunks có metadata (intent, subtype, section\_id, step, year...)

phase2\_embedding.py

Tạo embedding vector (Gemini REST API), lưu vào vector DB (vector\_db.json)

phase4\_prompt\_engineering.py

Xây dựng system prompt + context prompt cho LLM từ chunks retrieved

phase5\_llm\_generation.py

Deterministic formatter -> Groq LLM fallback -> Gemini naturalizer -> validate -> cache

automation\_ai\_rag.py

Gọi Groq API để refine query, phân loại topic, AI chunking

automation\_retriever.py

Phân tích query (intent, entities, query\_frame) -> vector search -> rerank -> enrich hierarchy

automation\_full\_pipeline.py

Chạy toàn bộ pipeline: extract -> AI chunk -> embed -> lưu vector DB

response\_cache.py

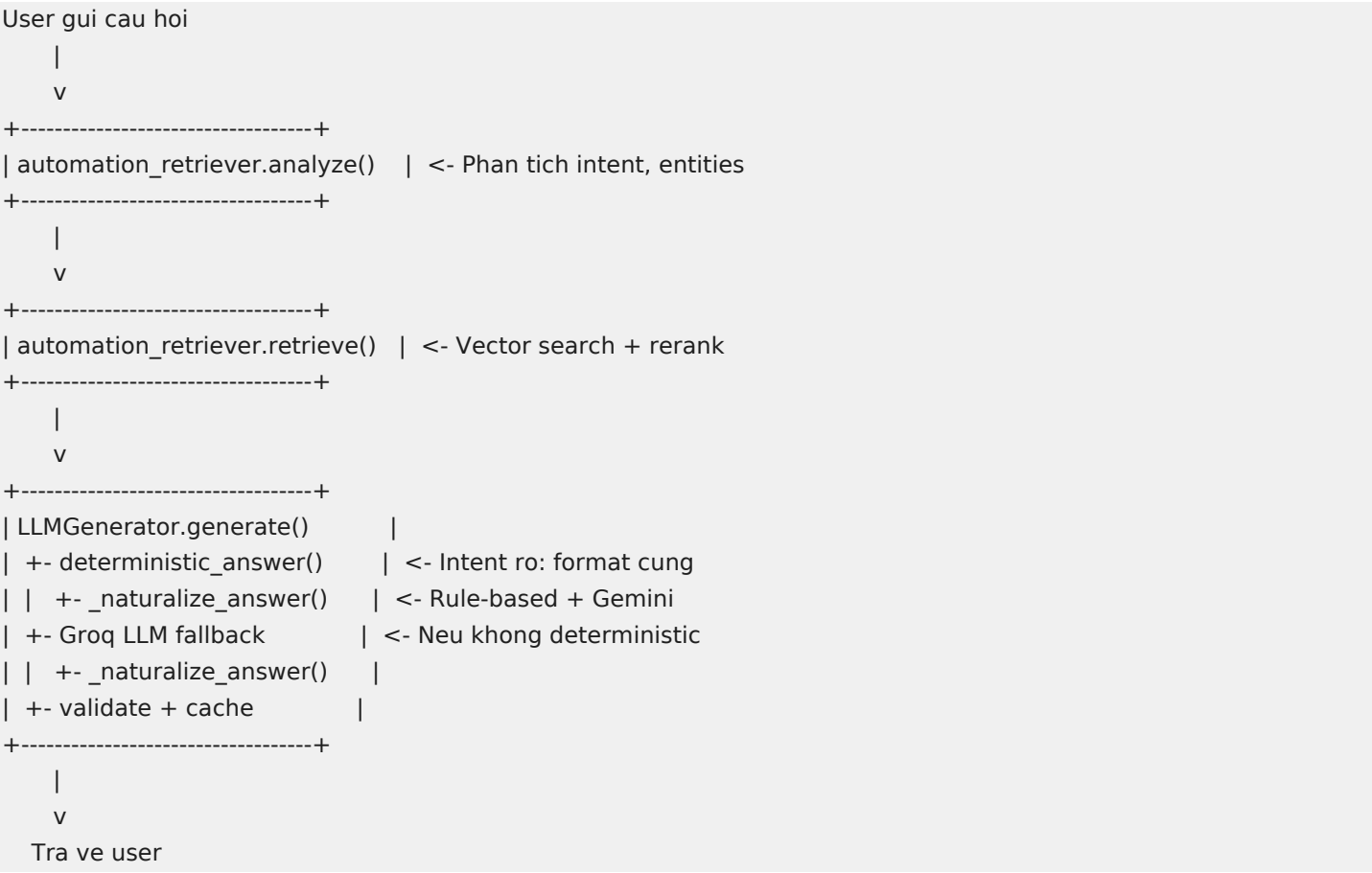
Cache câu trả lời theo query+intent+session, template fallback khi API lỗi

app.py

Flask web server, endpoint /chat, /chat\_stream, quản lý session/history

### 3. Luồng xử lý runtime

Khi user gửi câu hỏi qua web interface, hệ thống xử lý theo luồng sau:



## 4. Quy trình chạy project lần đầu

---

### Bước 1: Cài đặt

```
python -m venv .venv
.venv\Scripts\activate    # Windows
pip install -r requirements_rag.txt
pip install -r requirements_web.txt
```

### Bước 2: Cấu hình .env

```
GROQ_API_KEY=gsk_...
GEMINI_API_KEY=Alza...
GEMINI_MODEL_EMBED=gemini-embedding-001
GEMINI_MODEL_TEXT=gemini-1.5-flash
NATURALIZE_ENABLED=true
NATURALIZE_MAX_CHARS=2000
ADMISSION_YEAR=2025
```

### Bước 3: Chạy pipeline xây dựng dữ liệu

```
python automation_full_pipeline.py
```

Kết quả: all\_chunks.json + vector\_db.json được tạo.

### Bước 4: Khởi động web

```
python app.py
```

Truy cập: <http://localhost:5000>

### Bước 5: Chạy test

```
python -m unittest discover -s tests -p "test*.py"
```

## 5. Câu hỏi trọng tâm thuyết trình

---

### 1. Tại sao dùng RAG thay vì fine-tune LLM?

Dữ liệu nhập học thay đổi hàng năm (ngày, phí, ngành). RAG cho phép cập nhật bằng cách thay file TXT nguồn và chạy lại pipeline, không cần train lại model. Ngoài ra RAG đảm bảo câu trả lời có nguồn trích dẫn ([SOURCE]), giảm hallucination.

### 2. Deterministic formatter là gì? Tại sao không gọi LLM cho mọi câu hỏi?

Với các câu hỏi có intent rõ ràng (học phí, lịch theo ngành, các bước nộp hồ sơ), hệ thống trả lời bằng template cứng từ metadata chunk - không gọi API.

Lợi ích:

- Tốc độ nhanh (0ms API latency)
- Không tốn quota
- Đảm bảo 100% chính xác số liệu
- LLM chỉ dùng khi câu hỏi phức tạp/mở

### 3. Fact-lock validation hoạt động thế nào?

Trước và sau mỗi lần rewrite (rule-based hoặc Gemini), hệ thống extract "locked tokens": ngày tháng, số tiền, URL, B1-B4, PHẦN 1-4, [SOURCE]. Nếu output mới thiếu bất kỳ token nào hoặc thêm token nhạy cảm mới thì rollback về bản gốc. Đảm bảo style layer không bao giờ làm sai thông tin.

### 4. Gemini naturalizer đóng vai trò gì? Có rủi ro không?

Gemini là "style layer" cuối cùng - chỉ viết lại văn phong cho tự nhiên hơn, không thêm/bớt nội dung.

Rủi ro được kiểm soát bằng:

- Prompt chặt: cấm thêm thông tin, cấm xóa số liệu
- Temperature thấp (0.2)
- Fact-lock validation sau khi nhận output
- Rollback tự động nếu vi phạm
- Có thể tắt qua NATURALIZE\_ENABLED=false

### 5. Hệ thống phân biệt "bước" global vs local thế nào?

Global: "bước 1/2/3/4 của thủ tục nhập học" -> PHẦN 1-4 (4 phần lớn)

Local: "bước 1/2/3/4 nộp hồ sơ" -> B1-B4 (4 bước con trong PHẦN 4)

Hệ thống dùng context clues: nếu query chứa "hồ sơ", "phần 4", "nộp" thì local scope. Nếu chứa "thủ tục nhập học", "toàn bộ" thì global scope. Follow-up resolution dùng chat history để xác định axis đang nói.

### 6. Cache hoạt động ra sao?

Cache key = hash(normalized\_query + intent + session\_id + user\_id + style\_id). Khi cùng user hỏi lại câu

tương tự trong cùng session thì trả cache ngay, không gọi API. Cache có TTL, có `clear_session()` khi reset, và template fallback khi cả API lẫn cache đều miss.

## 7. Nếu Groq/Gemini API bị lỗi hoặc hết quota thì sao?

Hệ thống có 4 lớp fallback:

1. Deterministic formatter (không cần API)
2. Response cache (câu trả lời cũ)
3. Template response (câu trả lời mẫu theo intent)
4. Thông báo lỗi thân thiện

Phần lớn câu hỏi phổ biến (được deterministic formatter xử lý) nên bot vẫn hoạt động tốt khi mất API.

## 8. Embedding dùng model gì? Tại sao không dùng sentence-transformers local?

Dùng Gemini Embedding API (gemini-embedding-001) qua REST.

Lý do:

- Không cần GPU/RAM lớn trên server
- Chất lượng embedding tốt cho tiếng Việt
- Đã có sẵn GEMINI\_API\_KEY cho naturalizer
- Deployment nhẹ hơn (không cần tải model ~400MB)

## 9. Hệ thống xử lý out-of-domain (câu hỏi ngoài phạm vi) thế nào?

Khi intent == 'general' và không match bất kỳ admission topic nào, app.py trả domain guard message: "Mình chỉ hỗ trợ tư vấn thủ tục nhập học...". Không gọi LLM cho câu hỏi ngoài phạm vi, tiết kiệm quota và tránh hallucination.

## 10. Làm sao đảm bảo chất lượng output? Có test tự động không?

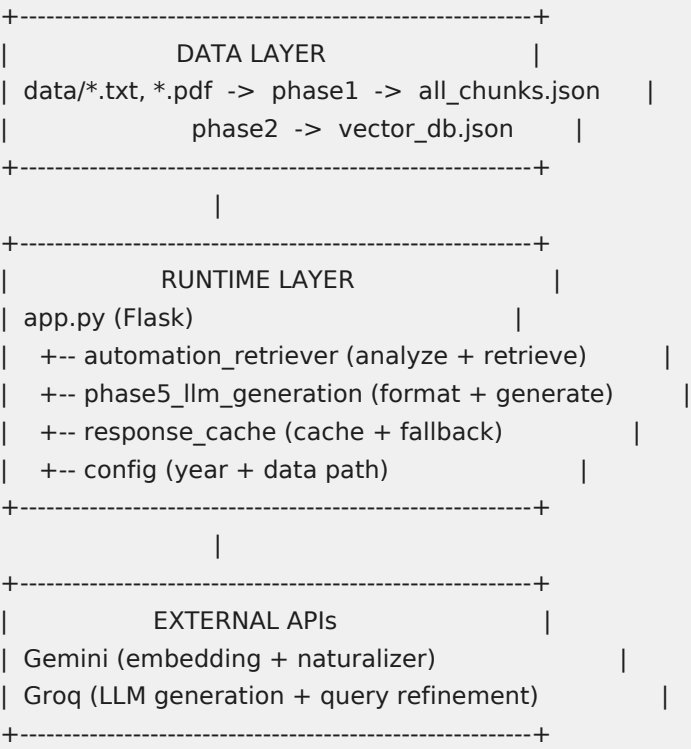
Có 352 unit tests bao gồm:

- Golden answer tests: kiểm tra output deterministic giữ đúng số liệu, format, source
- Fact-lock tests: kiểm tra Gemini naturalizer bị reject khi mất ngày/thêm tiền
- Step/section resolution tests: kiểm tra phân tích "bước"/"phần" đúng context
- Follow-up tests: kiểm tra hội thoại nhiều lượt giữ đúng axis

Chạy: `python -m unittest discover -s tests -p "test*.py"`



## 6. Sơ đồ kiến trúc tổng thể



Data Layer: Dữ liệu nguồn được xử lý offline thành chunks và vector DB.

Runtime Layer: Flask server xử lý request realtime, gọi retriever và generator.

External APIs: Gemini cho embedding/naturalizer, Groq cho LLM generation.